# AlphaNIST: 분석 리포트

게시용 버전 · 원문 https://arxiv.org/abs/2610.13354 · arXiv:2610.13354v1 · NVIDIA, University of Washington (Zhang, Akinola, Scalise, Guaman Castro, Yin 외) · 공식 코드 미공개 · 함께 읽은 논문 SGS arXiv:2610.12465 · 분석일 2026-10-28 · 논문 그림 © 저자, CC BY 4.0

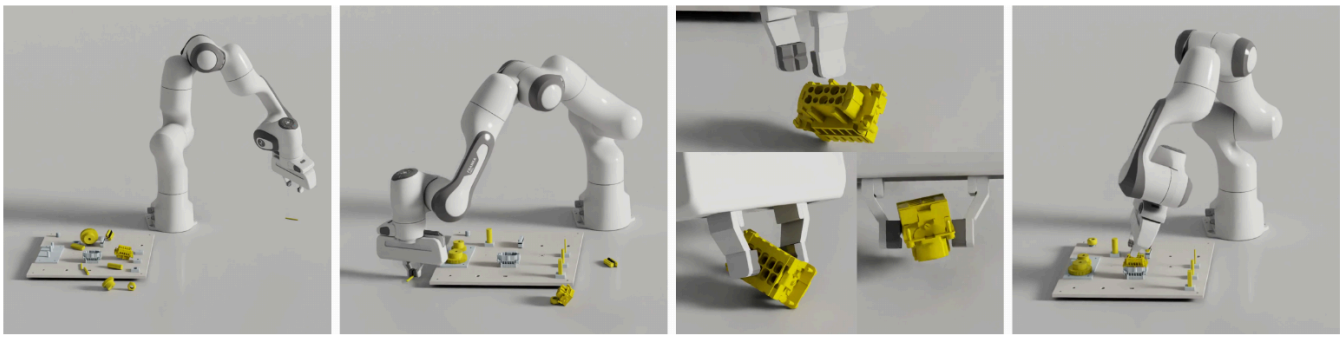


Fig. 1. **Emergent multi-component assembly with a single policy**. Trained with a binary task reward, AlphaNIST learns to select, grasp, reorient, and assemble diverse components, while autonomously sequencing multiple assemblies within a single episode. Shown are snapshots from a ten-component assembly sequence.

논문 Fig. 1: 이진 과제 보상으로 학습한 정책 하나가 부품 10개를 순서 지정 없이 고르고, 잡고, 돌리고, 조립한다.

## 핵심 요약

1. **무엇을 하는가:** NIST 조립 보드의 20개 과제를 Franka + 평행 그리퍼, PPO 정책 하나로 학습한다. 보상은 1 mm 성공 판정 하나(와 제어 비용)이고, OmniReset식 reset bank와 SGS식 성공률 기반 샘플링이 학습을 가능하게 한다. 학습 성공률 98.2%, 10쌍 연속 조립 84%.
2. **공유 정책의 우위는 확인된다:** 그림 6을 판독하면 같은 25K iteration에서 공유 98.3% 대 분리 81.4%이고, 분리 정책은 50K까지 돌려도 88.1%다. 실패는 작은 부품 과제(M4 너트, 4 mm 막대·페그, RJ45)에 몰려 있다.
3. **비교의 기준이 섞여 있다:** 그림 5(b)의 분리 dense 막대(78.3%)는 50K 값이고, "39.3배 계산량"은 50K까지 돈 분리 정책의 비용이다. 같은 25K 기준이면 약 20배다. 그림 7은 iteration을 고정하고 env를 256배 늘린 것이라 계산량을 맞춘 비교가 아니다.
4. **98.2%는 쉬운 시작점이 섞인 평균이다:** 분리 정책이 무작위 초기 상태에서 0.3% 이하인 과제도 heatmap에서는 31-52%로 나온다. 테이블에 놓인 부품에서 시작했을 때의 성공률은 따로 보고되지 않았다.
5. **"이진 > dense"는 이 dense 변형에 한정된다:** dense 비교군은 OmniReset의 계수(실효 0.01)를 성공 시 종료 구조에 붙였다. OmniReset 구현은 성공 보상을 매 step 주고 종료하지 않는다. 성공 시 종료 + 양의 보조 보상에서는 $s/(1-\gamma) > R$이면 완료를 끝까지 미루는 것이 최적이다(toy).
6. **SGS를 그대로 옮기지 않았다:** 식 4의 커널(지수 ½)은 SGS 원 설정(온도 2, 지수 ¼)과 다르고, $\varepsilon_q \cdot L \cdot M$이 없어 학습 후반의 배분이 22-99.6%로 정해지지 않는다. SGS의 보상을 "작업별 파지 shaping"이라고 한 서술도 SGS 원문과 다르다.
7. **구현 가능성:** 공식 코드는 없다. 독립 MuJoCo 구현(MIT, 사전학습 checkpoint 포함)이 비슷한 성공률을 보고하지만, 접촉 플러그인이 x86-64 전용이라 이 맥에서는 이식이 필요하다. 이 맥에서는 샘플러·보상 toy 검증까지가 현실적이다.

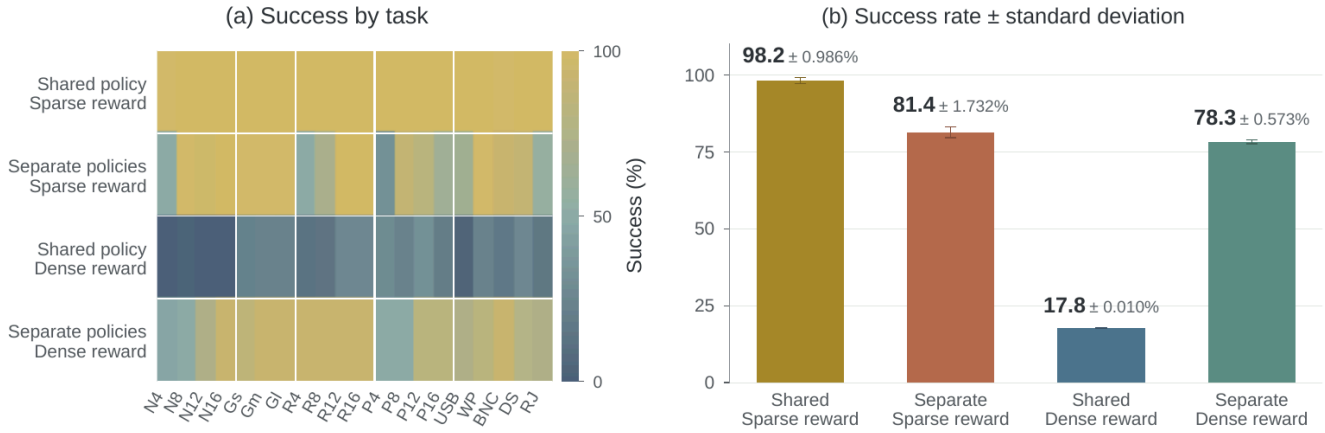



Fig. 5. **Training across heterogeneous assembly pairs substantially improves learning from a sparse binary task reward.** At 64K environments across 25k iterations: (a) compares 20-task success rates for one shared policy versus 20 separately trained policies. The shared policy (first row) achieved above 98% success rate across all tasks, including three precision assemblies that corresponding specialists rarely solve. Dense shaping does not exhibit the same benefit. Panel (b) shows overall success, note that the separate-policy success rate is average across all 20 per-task policies; *Asset notation:* N, G, R, and P denote nuts, gears, rods, and pegs, respectively; numeric suffixes indicate nominal diameters in millimeters, while s, m, and l indicate gear size. "Sparse reward" denotes Eq. (6); "dense reward" additionally includes the shaping terms in Eq. (7).

논문 Fig. 5: 64K env에서 공유 정책과 분리 정책 20개, sparse와 dense 보상의 과제별·전체 성공률.

# 01. 논문 분석: AlphaNIST (arXiv:2610.13354)

- 제목: *AlphaNIST: Emergent Assembly Skills from Binary Task Rewards*
- 저자: Octi Zhang, Iretiayo Akinola, Rosario Scalise, Mateo Guaman Castro, Patrick Yin, Tobias Widmer, Miles Macklin, Chris Amevor, Dylan Turpin, Dirk Van Gelder, Jan Carius, Ankur Handa, Gavriel State, Yunrong Guo, Yashraj Narang (NVIDIA, University of Washington)
- 공개: arXiv v1 2026-10-09 (cs.AI, cs.RO), "8 pages, 9 figures, ICRA" · 프로젝트 https://alpha-nist.github.io · 코드 미공개("심사·수정 기간 뒤 공개 예정", Sec. I)
- 라이선스: 논문 **CC BY 4.0** (arXiv abs 페이지에서 확인)
- 함께 읽은 논문: SGS, *A Balanced Data Diet: Addressing the Exploration Bottleneck in Mega-Scale RL for Robot Control* (arXiv:2610.12465v1, CoRL 2026, CC BY 4.0). AlphaNIST가 시작 상태 샘플링을 가져온 [7]입니다.
- 분석 범위: 본문 8쪽(부록 없음), 프로젝트 페이지, SGS 논문 중 샘플러·보상·평가 부분, OmniReset 공개 구현(UW-Lab/UWLab @ `2b40afb`)의 보상·PPO 설정, 독립 MuJoCo 구현(suniique/assembly-nist @ `eb5d15e`)

---

## 1. Abstract (한 줄 요약과 주장)

**한 줄 요약:** NIST Assembly Task Board #1의 조립 과제 20개(너트 체결, 기어 끼우기, 막대·페그 삽입, 커넥터)를 Franka 팔 하나와 평행 그리퍼로, **이진 성공 보상 하나와 PPO 정책 하나**로 시뮬레이션에서 학습합니다. 시연, 진행도 보상, 스크립트 동작, 과제별 조정 없이 학습 성공률 98.2%를 보고합니다.

논문이 내세우는 기여는 세 가지입니다(Sec. I).

1. 다중 과제 RL 틀을 제시합니다. 구성 요소는 (A) 이질적인 조립 쌍에 걸친 reset 상태 cache(OmniReset[10]), (B) cache 안에서의 success-guided sampling(SGS[7]), (C) 공통 이진 과제 보상, (D) 다중 GPU 대규모 학습입니다.
2. NIST ATB1의 20개 과제를 모두 푸는 최초의 다중 과제 RL 정책이라고 주장합니다. 점군을 입력으로 받고, 부품 초기 자세를 넓게(뒤집힌 상태 포함) 뽑습니다. 분리 학습한 전문가 정책보다 성공률이 높아 **긍정적 전이**가 있다고 합니다.
3. 조립 쌍 최대 10개를 순서 지정 없이 한 episode에서 이어서 푸는 최초의 조립 정책이라고 주장합니다. 계층 구조나 과제 분해 없이 순서 정하기 행동이 나온다는 것입니다.

---

## 2. Method

### 2.1 전체 구조 (Sec. IV, Algorithm 1)

```
reset bank D = {(e_i, s̄_i)}   e: 조립 쌍(20종), s̄: 로봇·보드·부품 상태      (Eq. 3)
   reset 종류: 놓인 부품 · 집기 · 운반 · 끼우기 직전 · 부분 조립 (OmniReset[10] 방식, Fig. 4)
        │  항목별 최근 L개 결과의 성공률 p̂_i
        ▼
Beta 샘플링: q_i ∝ ε_q + √(p̂_i (1 − p̂_i))   (Eq. 4)   ← SGS[7]
        │  병렬 환경마다 (e, s₀) 배정, 과제 ID는 관측에 없음
        ▼
공유 정책 π_θ(a | o): 부품·짝 형상·로봇 표면 점군 + 고유감각 → Δq(7) + 그리퍼(1), 25 Hz
        │  표준 PPO, actor·critic 분리 SimBa 망(Fig. 3)
        ▼
보상 r = 1[d_asm < 1 mm] − c(‖a‖² + ‖q̇‖² + ‖τ‖²)   성공 시 1을 한 번 주고 종료   (Eq. 5–6)
```

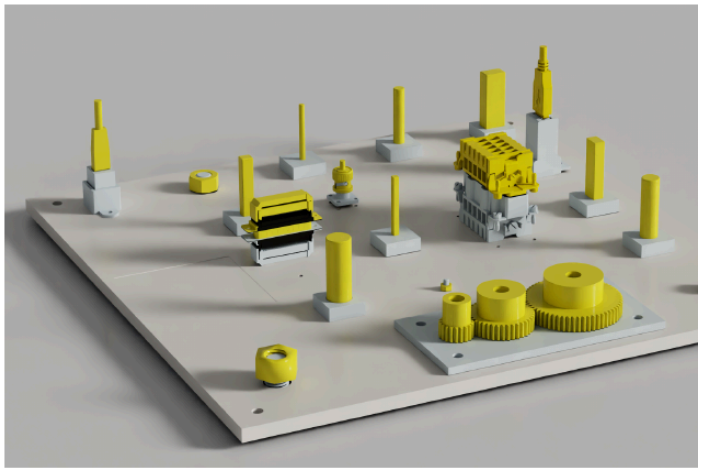


Fig. 2. **The complete NIST Assembly Task Board #1 in** Yellow indicates the 20 movable parts in their fully-assembled

논문 Fig. 2: 시뮬레이션의 NIST Assembly Task Board #1. 노란색이 움직이는 부품 20개(완전 조립 상태).

### 2.2 문제 설정 (Sec. III, Eq. 1-2)

20개 조립 과제를 공유 정책 하나의 유한 horizon MDP로 둡니다. 각 episode 시작에 (과제 e, 초기 상태 $s_0$) ~ ρ를 뽑고, 정책은 부분 관측 o_t만 봅니다. 과제 차이는 **이산 과제 ID가 아니라 표면 점군**으로만 드러납니다(Sec. III-A).

### 2.3 Reset bank (Sec. IV-B, Eq. 3)

OmniReset[10]의 다양한 reset을 이질적 조립에 맞게 바꿔, 각 reset 상태에 조립 정체(e_i)를 함께 저장합니다. bank는 놓인 부품, 집기, 운반, 끼우기 직전, 부분 조립 상태를 포함합니다(Fig. 4). 논문은 "이 reset 종류는 학습 분포의 영역일 뿐 정해진 순서가 아니며, 정책은 단계 식별자를 받지 않는다"고 강조합니다. bank 크기 M과 종류별 비율은 적혀 있지 않습니다.

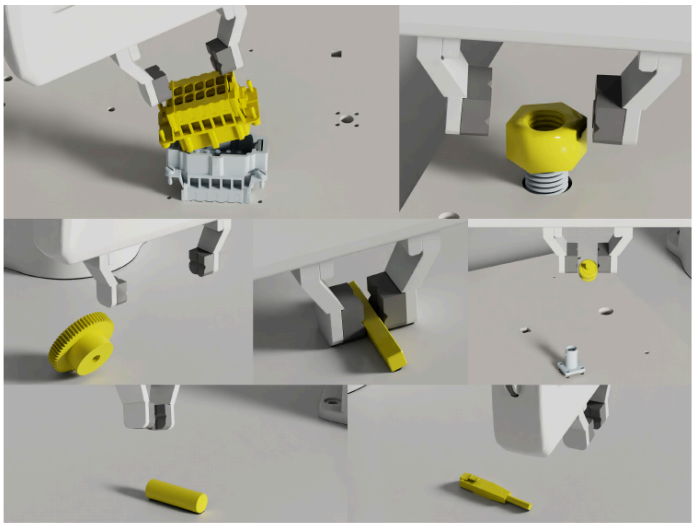


Fig. 4. **Diverse assembly reset states.** Examples of resets across tasks, ranging from random object poses to in-hand grasps and partial assemblies.

논문 Fig. 4: 과제별 reset 예시. 무작위로 놓인 부품부터 손에 쥔 상태, 부분 조립까지.

### 2.4 Beta 샘플링 (Sec. IV-B, Eq. 4)

항목 i의 최근 L개 episode 성공률을 $\hat{p}_i$라 할 때, 다음 확률로 고릅니다.

```
q_i ∝ ε_q + √(p̂_i (1 − p̂_i))
```

√(p(1−p))는 베르누이 표준편차입니다. 성공률이 중간인 항목을 자주 뽑고, ε_q > 0으로 모든 항목이 계속 뽑힐 수 있게 합니다. 균등 샘플링 비교군은 q_i = 1/M으로 두고 나머지를 그대로 둡니다. **ε_q와 L 값은 적혀 있지 않습니다.**

논문은 "SGS의 샘플링 규칙을 유지한다"(Sec. II-A)고 씁니다. 하지만 SGS 원 논문의 조작 설정(κ = 1, t = 0.5, ε = 1e-4, 온도 T = 2)을 풀면 q ∝ (p(1−p))^¼입니다. 식 4의 지수 ½과 다르고, 이 차이 때문에 학습 후반에 표본이 배분되는 방식이 크게 달라집니다(`03` §1).

### 2.5 보상 (Sec. IV-C, Eq. 5-7)

- **이진 과제 보상(Eq. 5):** 부품의 현재 상대 자세와 완전 조립 자세 사이 거리 d_asm이 ε = 1 mm보다 작으면 1, 아니면 0입니다. 성공 보상은 한 번만 주고 episode를 끝냅니다. 진행 중에는 과제 보상이 없습니다.
- **제어 비용(Eq. 6):** −c(‖a_t‖² + ‖q̇_t‖² + ‖τ_t‖²)이고, c는 과제 공통입니다. c 값은 적혀 있지 않습니다.
- **종료:** 조립 성공, horizon T, 또는 팔 관절이 하드웨어 속도·토크 한계를 넘을 때입니다. 다중 부품 episode에서는 모든 활성 쌍이 조립되어야 성공 보상이 나옵니다.
- **dense 비교군(Eq. 7):** r + c₂ r_reach + c₃ r_align이고, c₂ = c₃ = 0.01은 "OmniReset의 계수와 같다"고 합니다. r_reach는 엔드이펙터가 부품 중심에 가까울수록, r_align은 목표 조립 방향에 맞을수록 커집니다. 함수 꼴은 적혀 있지 않습니다.

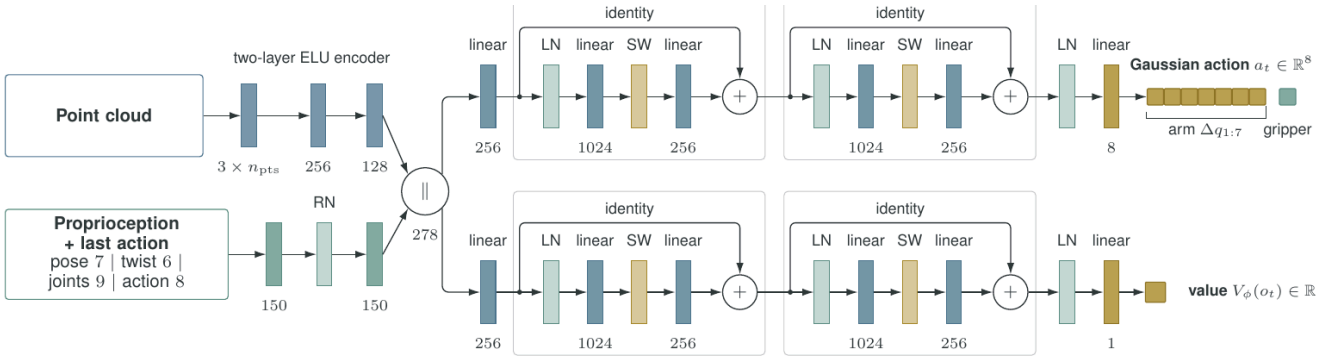



Fig. 3. **Policy architecture.** Actor and critic have the same architecture with separate parameters. Inputs are drawn once; each network contains two consecutive residual blocks. RN: running normalization; LN: layer normalization; SW: Swish activation; $n_{\text{pts}}$: number of input points.

논문 Fig. 3: actor와 critic은 같은 구조, 다른 파라미터. 점군 인코더 + 고유감각 → SimBa residual block 2개.

### 2.6 정책과 제어 (Sec. IV-D, Fig. 3)

- actor와 critic은 같은 구조에 파라미터만 따로 둔 SimBa[28] 망입니다. 점군 인코더(3 × n_pts → 256 → 128, ELU)와 고유감각 150차원(running normalization)을 이어 278차원을 만들고, 폭 256·확장 1024의 residual block 2개를 지납니다.
- 150은 [자세 7, twist 6, 관절 9, 직전 행동 8] = 30의 5배라, 고유감각 5프레임 이력으로 보입니다(가설, `03` §6).
- 로봇은 식별된 관성·구동계를 쓴 7-DoF Franka와 평행 그리퍼(구동 손가락 1 + 수동 결합 손가락 1)입니다. 25 Hz로 관절 위치 상대 목표 Δq(7)와 그리퍼 명령(1)을 냅니다. libfranka 강성·감쇠의 implicit impedance 제어기를 쓰고, 속도 한계는 2.175 / 2.61 rad/s, 토크 한계는 87 / 12 N·m입니다.

### 2.7 분산 학습 (Sec. IV-E)

동기식 PPO를 1K-256K 환경으로 늘리고, 다중 GPU에서는 GPU당 4K 환경을 둡니다. iteration마다 환경당 32 step을 모으고, 5 epoch × minibatch 4개로 최적화합니다. 이 세 값은 OmniReset 공개 설정(`rsl_rl_cfg.py`: 32 step, 5 epoch, minibatch 4, γ = 0.99, λ = 0.95, lr 1e-4)과 같습니다. 그래서 AlphaNIST도 나머지 PPO 값을 OmniReset에서 가져왔을 가능성이 있지만, 논문에는 없습니다.

**논문에 없는 설정값:** γ, λ, 학습률, clip, entropy 계수, c, ε_q, L, M, n_pts, horizon T(본문 "15초"에서 375 step으로 추정), reset 종류별 비율, GPU 종류.

---

## 3. Experiments

### 3.1 설정 (Sec. V)

- **질문:** (Q1) 이진 보상에서 공동 학습과 과제별 분리 학습의 비교, (Q2) 병렬 경험과 reset 샘플링의 영향, (Q3) 이진 보상과 dense 진행도 보상의 비교입니다. 다중 부품 조립과 창발 행동은 추가 관찰입니다.
- **이질성 비교:** 공유 정책 하나와 과제별 정책 20개를 비교합니다. "실행당 상호작용 예산을 맞추고 나머지는 같게" 했고, 실행마다 64K 환경입니다.
- **규모 평가(Fig. 7):** GPU 수, 환경 수, rollout 배치를 함께 늘리고, 25K iteration checkpoint를 평가합니다. 평가는 확률적 행동으로 하고, 공통 고정 혼합 reset bank에서 과제 안 균등 샘플링으로 시작합니다. 시드 5개 × 과제당 32회 × 과제 20개 = 점마다 3,200회입니다.
- **성공 판정:** 15초 안에 완전 조립하고 안전 위반이 없어야 합니다. "모든 실험에서 성공률은 모든 reset 구성을 포함한다"(Sec. V)고 적혀 있습니다.

### 3.2 Q1: 이질적 조립 쌍의 공동 학습 (Fig. 5, Fig. 6)

| 조건 (64K env) | 논문 값 (Fig. 5b) | 그림 6 판독: 25K | 그림 6 판독: 50K |
|---|---|---|---|
| 공유 정책, sparse | 98.2 ± 0.986% | 98.3 | 99.9 |
| 분리 20개, sparse | 81.4 ± 1.732% | 81.4 | 88.1 |
| 공유 정책, dense | 17.8 ± 0.010% | 17.9 | 18.2 |
| 분리 20개, dense | 78.3 ± 0.573% | 70.9 | 78.3 |

판독은 `analysis/fig_digitize.py`로 했습니다. 그림 5(a) heatmap 80칸을 색 막대로 역산한 행 평균(99.9 / 81.9 / 17.1 / 79.0)과, 본문의 칸 값(분리 dense M4 45.0%, 중간 기어 93.6%)이 ±0.7%p 안에서 재현되어 판독 정확도를 확인했습니다.

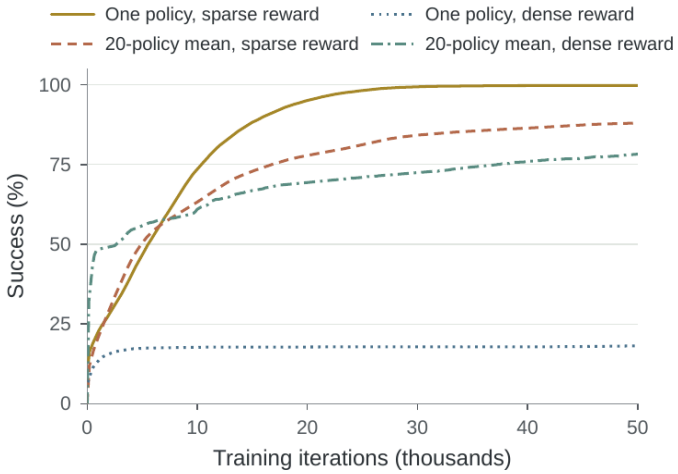



Fig. 6. **Joint sparse training converges faster than separate policies.** Curves compare one shared policy trained on all 20 tasks with the mean of 20 policies trained separately, one per task.

논문 Fig. 6: 공유 정책 하나 대 분리 정책 20개의 평균, 50K iteration까지.

해석 (재계산은 `04` C1-C6):

- **공유 정책의 우위는 판독으로도 확인됩니다.** 같은 25K에서 98.3% 대 81.4%이고, 분리 정책은 50K까지 돌려도 88.1%입니다.

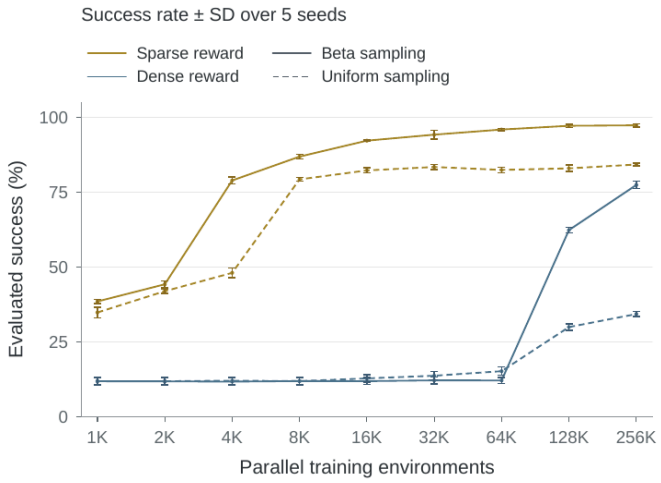
- **그림 5(b)는 checkpoint가 섞여 있습니다.** 캡션은 "25k iterations"인데, 분리 dense 막대 78.3%는 그림 6의 50K 값입니다(25K에서는 70.9%). 본문도 "최종 checkpoint에서 78.3%"라고 씁니다.
- **"39.3배"는 iteration 수가 다른 비교입니다.** 분리 정책 20개의 총 16.1K GPU-h를 나누면 정책 하나당 805 GPU-h로, 공유 정책(410 GPU-h, 25K iteration)의 1.96배입니다. 분리 정책이 50K까지 돈 비용으로 보입니다. 같은 25K 기준이면 약 20배이고, 이때 분리 정책 성공률은 81.4%입니다(가설: 비용이 iteration에 비례).
- **분리 정책의 실패는 몇 과제에 몰려 있습니다.** 판독값으로 분리 sparse에서 60% 미만인 칸은 N4 50.0, R4 51.7, P4 31.3, RJ 56.9이고, 나머지 16개 과제의 평균은 90.4%입니다. 같은 칸에서 공유 정책은 99.4-100입니다.
- **본문의 "0.3% 이하"와 heatmap이 다릅니다.** 본문은 분리 정책이 M4 너트, 4 mm 페그, 막대 삽입에서 0.3% 이하라고 쓰지만(Sec. V-A), heatmap의 해당 칸은 50%, 31%, 52%입니다. 본문 문장은 "무작위 초기 상태에서"라는 조건이 붙어 있으므로, heatmap은 쉬운 reset(손에 쥔 상태, 끼우기 직전, 부분 조립)이 섞인 평균으로 읽힙니다. 그렇다면 이 과제들의 bank에서 그런 쉬운 시작점이 30-50%를 차지하고, 98.2%라는 헤드라인도 같은 혼합 분포의 평균입니다(`03` §3).
- **dense 보상은 분리 학습에서는 통합니다.** 분리 dense 정책은 M4 너트에서 45%로, 분리 sparse의 무작위 초기 상태 성공률(0.3% 이하)보다 오히려 낮습니다. dense의 실패는 공유 학습에서만 극적입니다(17.8%). 논문의 해석은 "dense 항이 형상마다 다른 부분 해를 선호한다"는 것입니다.

### 3.3 Q2: 병렬 경험과 샘플링 (Fig. 7)

| 조건 | 1K | 4K | 8K | 256K |
|---|---|---|---|---|
| sparse + Beta | 38.6 | 78.9 | 86.9 | 97.4 |
| sparse + 균등 | 약 35 | 약 48 | 약 79 | 84.3 |
| dense + Beta | 약 12 | 11.8-12.0 | 약 12 | 77.4 |
| dense + 균등 | 약 12 | 약 12 | 약 12 | 34.3 |

"약"은 그림에서 눈으로 읽은 값(±1)이고, 나머지는 본문 값입니다.

- **Beta와 균등의 차이는 평가 잡음보다 훨씬 큽니다.** 256K에서 13.1%p 차이인데, 3,200회 기준 이항 표준오차는 0.3-0.6%p입니다. 다만 SD는 평가 시드 5개의 것이고, 학습 시드는 설정마다 하나로 보입니다.
- **계산량이 고정된 비교가 아닙니다.** iteration은 25K로 고정하고 환경 수를 1K에서 256K로 늘리므로, 오른쪽 끝은 env step이 256배($2.05\times10^{11}$)이고 GPU가 64장입니다(`04` C8). "sparse 보상이 규모에서 이득을 본다"는 결론은 맞지만, 같은 계산량에서 무엇이 나은지는 이 그림으로 알 수 없습니다.
- **dense는 64K까지 12% 근처에 머물다가 128K 이후 오릅니다.** 256K에서는 Beta와 함께 77.4%에 이릅니다. 즉 dense도 규모가 충분하면 학습되며, 이진 보상이 우월한 폭은 규모가 커질수록 줄어듭니다(64K에서 약 84%p, 256K에서 20%p).




논문 Fig. 7: 병렬 환경 수에 따른 평가 성공률(25K iteration). 색은 보상, 실선·점선은 Beta·균등 샘플링.

### 3.4 Q3: 보상 형식

"모든 규모에서 이진 과제 목표가 평가한 dense shaping보다 낫다"(Sec. VII)는 결론은 Fig. 7의 비교에서 나옵니다. 다만 dense 비교군의 구조를 따져 볼 필요가 있습니다. 이 비교군은 OmniReset의 dense 보상을 그대로 쓴 것이 아니라, OmniReset의 보조 계수(0.01)를 "성공하면 1을 한 번 주고 종료"하는 AlphaNIST 구조에 붙인 것입니다.

OmniReset 구현은 성공 상태에 머무는 동안 매 step 성공 보상을 주고, 성공으로는 종료하지 않습니다(`04` C14). 반면 매 step 양의 보조 보상이 있고 성공 시 종료하면, $s/(1-\gamma) > R$일 때 **완료를 horizon 끝까지 미루는 것이 최적**이 됩니다. toy에서 $\gamma = 0.99$, $s \approx 0.02$, $R = 10$이 이 조건에 해당했습니다(`03` §2). 이 유인이 실제 학습에서 얼마나 작용했는지는 보조 항의 꼴과 $\gamma$가 논문에 없어서 판단할 수 없습니다. 하지만 "이진 > dense"라는 결론은 **이 dense 변형에 한정**해서 읽어야 합니다.

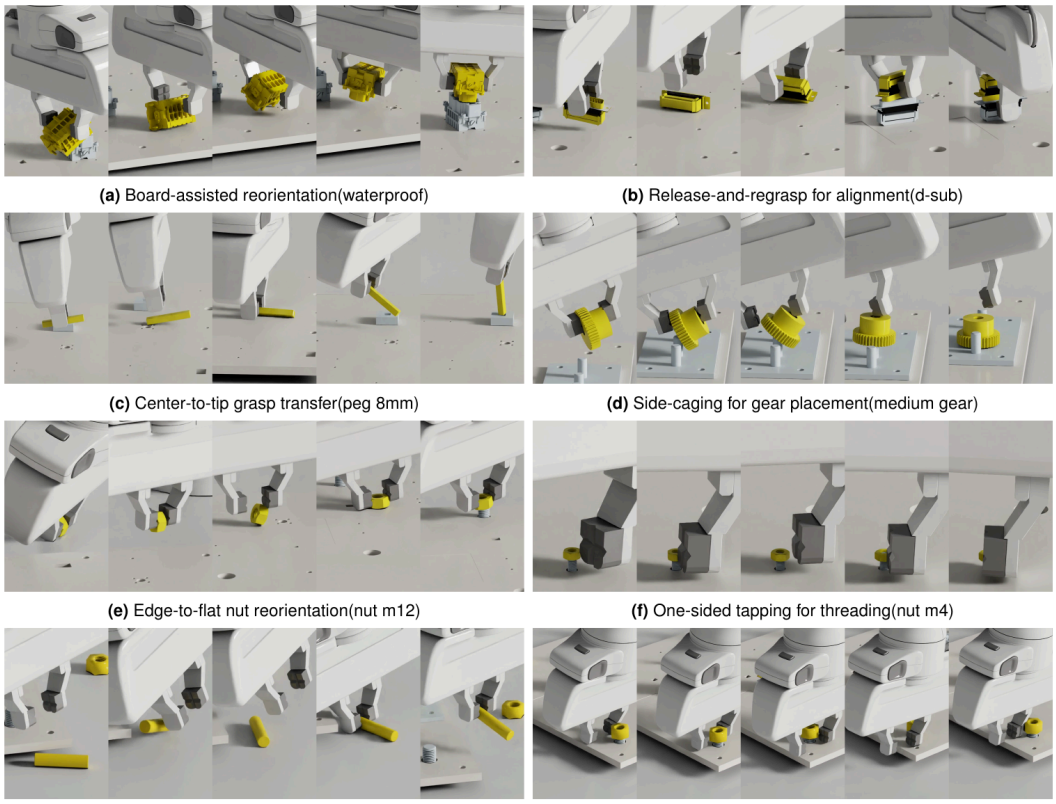
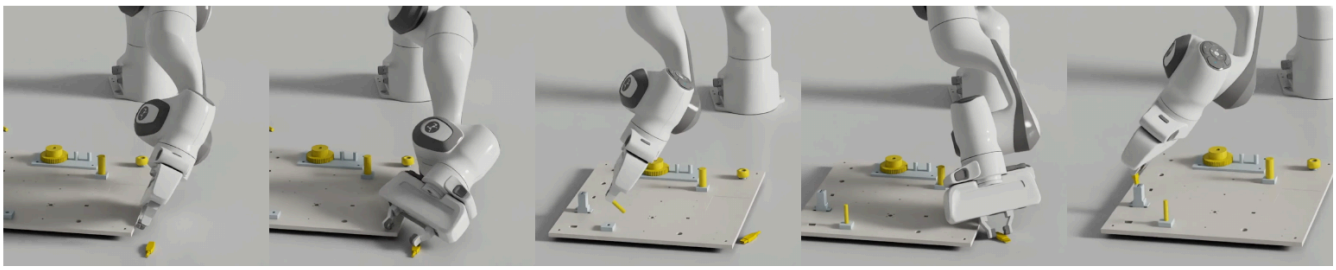



Fig. 8. **One shared policy discovers diverse contact strategies from a sparse binary task reward.** Each panel proceeds from left to right. (a) Waterproof connector: board contact helps reorient the part before insertion. (b) D-sub connector: the part is set down and regrasped for alignment with its socket. (c) 8 mm peg: the part is dropped from a center grasp and regrasped near one end, leaving its remaining length free for insertion. (d) Medium gear: the part pivots from a side grasp into a horizontal pose over its mounting post. (e) M12 nut: the nut is rolled from its edge onto a flat face before placement on the bolt. (f) M4 nut: repeated one-sided taps advance the nut along the thread. (g) 8 mm rod: the fingertip shovels the rod to a position where it is easier to lift. (h) M16 nut: the gripper moves around the nut, using fingertip contact to turn it along the bolt.

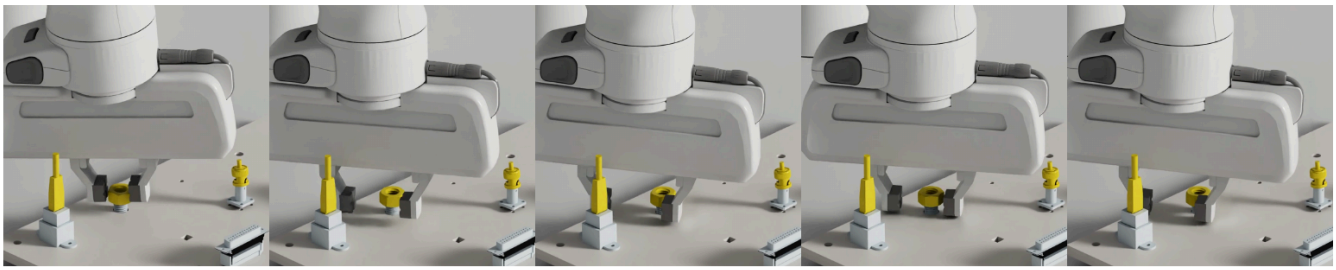
논문 Fig. 8: 이진 보상에서 나온 접촉 전략 여덟 가지.

### 3.5 창발 행동과 다중 부품 조립 (Fig. 8, Fig. 9, Table I)

- **단기 전략(Fig. 8):** 보드 접촉으로 커넥터 돌리기, 내려놓고 다시 잡기, 페그를 끝 쪽으로 다시 잡기, 옆에서 감싸 기어 세우기, 너트 굴려 눕히기, 한쪽 손끝 두드림으로 M4 너트 나사산 진행, 막대 밀어서 집기 쉬운 곳으로 옮기기, 그리퍼가 너트 둘레를 돌며 M16 체결.
- **장기 행동(Fig. 9):** USB-A를 가까이 옮겨 두고 8 mm 막대를 조립한 뒤 돌아와 삽입합니다. M12 체결 중에는 옆 RJ45를 피하려고 작게 회전합니다.
- **Table I:** 활성 쌍 1-10개의 성공률은 99, 97, 95, 93, 92, 90, 89, 87, 86, 84%입니다. 연속 비율이 0.977-0.989로 거의 일정해서, 쌍 하나를 더할 때마다 약 1.75%씩 줄어드는 기하 감소(적합 비율 0.9825)로 잘 설명됩니다. 같은 비율로 20개를 외삽하면 70.8%이고, 10개 정책을 두 번 쓰면 70.6%입니다(`04` C10).
- **보고 방식의 한계:** 행동의 빈도는 보고되지 않았고 영상 예시만 있습니다. Table I에는 시행 수, 표준편차, 활성 쌍을 고르는 방법이 없습니다. 활성 쌍 수마다 정책을 따로 학습했습니다(캡션). Fig. 1의 "single policy"는 10개용 정책 하나를 뜻합니다.



**(a)** Bring USB-A closer, assemble the 8 mm rod, then return



**(b)** Making small turns threading M12 to avoid RJ45 collision

Fig. 9. **Emergent coordination in multi-pair assembly.** (a) The policy brings the USB-A connector from a distant corner to a nearby working position, leaves it there while assembling the 8 mm rod, then returns to insert the connector. (b) During M12 nut threading, the end effector makes small turns to avoid touching the neighboring RJ45 connector. Each sequence proceeds from left to right.

논문 Fig. 9: 다중 부품 조립에서 나온 조정 행동(USB-A를 옮겨 두기, RJ45를 피해 M12 돌리기).

## 4. Limitations

### 4.1 저자가 밝힌 한계 (Sec. VI)

모든 결과가 시뮬레이션입니다. 하드웨어에 맞춘 속도·토크 한계는 지키지만, 인식 오차, 지연, 접촉 모델 불일치에 대한 강건성은 확인하지 않았습니다. 실제 보드에서 센서 관측과 접촉·지연 랜덤화로 평가할 예정입니다. 10개 정책을 이어 쓰면 보드 전체를 풀 수 있겠지만, 20개 다중 부품으로 넓히는 것은 과제로 남았습니다.

### 4.2 분석하면서 추가로 발견한 한계와 의문점

1. **헤드라인 98.2%는 reset 혼합 분포의 평균입니다.** 분리 정책이 무작위 초기 상태에서 0.3% 이하인 과제도 heatmap에서는 31-52%로 나오므로, 쉬운 reset이 상당한 비중을 차지합니다(§3.2). 논문은 공유 정책의 **reset 종류별 성공률**(특히 테이블에 놓인 부품에서 시작하는 경우)을 보고하지 않습니다. 반면 독립 MuJoCo 구현은 이 값을 보고합니다(놓인 부품 98.1%, 끼우기 직전 98.7% 등, `02` §5).
2. **그림 5(b)는 checkpoint가 섞여 있고, 39.3배는 iteration 수가 다른 비교입니다**(§3.2, `04` C2-C3).
3. **dense 비교군은 OmniReset의 보상 구조와 다릅니다.** 성공 시 종료하는 구조에 매 step 양의 보조 보상을 붙이면 완료를 미룰 유인이 생깁니다(§3.4, `03` §2). "이진이 dense보다 낫다"는 일반 결론의 근거로는 약합니다.
4. **SGS를 정확히 옮겼는지 불분명합니다.** 식 4의 지수(½)는 SGS 원 설정(¼)과 다르고, ε_q·L·M이 없어 샘플러 동작을 정할 수 없습니다(`03` §1). 또 본문은 SGS가 "작업별 보조 파지·그리퍼 shaping을 유지한다"(Sec. II-C)고 쓰지만, SGS v1 부록 A.3.2의 조작 보상은 성공·실패·행동 정규화 항뿐입니다(`04` C13). SGS는 Reaching 초기 분포에서 평가하므로(SGS Table 6), AlphaNIST의 혼합 분포 성공률과 바로 비교할 수 없습니다.
5. **학습 시드 반복이 없습니다.** 그림 5와 7의 ±는 평가 시드의 표준편차입니다. "17.8 ± 0.010%"는 시드당 1,460만 회가 필요한 값이라 단위 오기(1.0%)로 보입니다(`04` C6).
6. **그림 7은 계산량을 맞춘 비교가 아닙니다**(§3.3).
7. **관측은 시뮬레이터의 정답입니다.** 점군은 알려진 메쉬를 참 자세에 놓고 표면에서 뽑은 것이라, 사실상 완벽한 자세 추정과 형상 정보를 줍니다. "자세 추정 대신 점군"(Sec. I)이라는 장점은 실제 센서로 옮길 때 다시 검증해야 합니다.

8. **재현 정보가 부족합니다.** 코드가 공개되지 않았고, §2.7의 설정값과 reset bank 구성이 없습니다. 64K 환경 실행 하나가 16 GPU × 약 26시간(410 GPU-h)이고, GPU 종류는 적혀 있지 않습니다.

# 03. 핵심 식 해석과 toy 검증

스크립트는 모두 CPU에서 수 초 안에 끝납니다.

| 스크립트 | 내용 | 결과 |
|---|---|---|
| `analysis/fig_digitize.py` | 그림 5(a)·6 판독 (원문 PDF를 300 dpi로 렌더링) | `fig_digitize_results.json` |
| `analysis/sampling_toy.py` | 식 4와 SGS 커널 비교 | `sampling_toy_results.json` |
| `analysis/reward_toy.py` | 성공 시 종료 + 보조 보상의 최적 정책 | `reward_toy_results.json` |
| `analysis/table_check.py` | 본문 수치 재계산 | `table_check_results.json` |

## 1. 시작 상태 샘플링: 식 4와 SGS는 같은 규칙인가

### 1.1 식 정리

AlphaNIST 식 4는 $q_i \propto \varepsilon_q + \sqrt{\hat{p}_i(1 - \hat{p}_i)}$입니다.

SGS 원 논문(arXiv:2610.12465 식 1-2)의 조작 설정($\kappa = 1$, $t = 0.5$, $\varepsilon = 1e{-}4$, $T = 2$)은 다음과 같습니다.

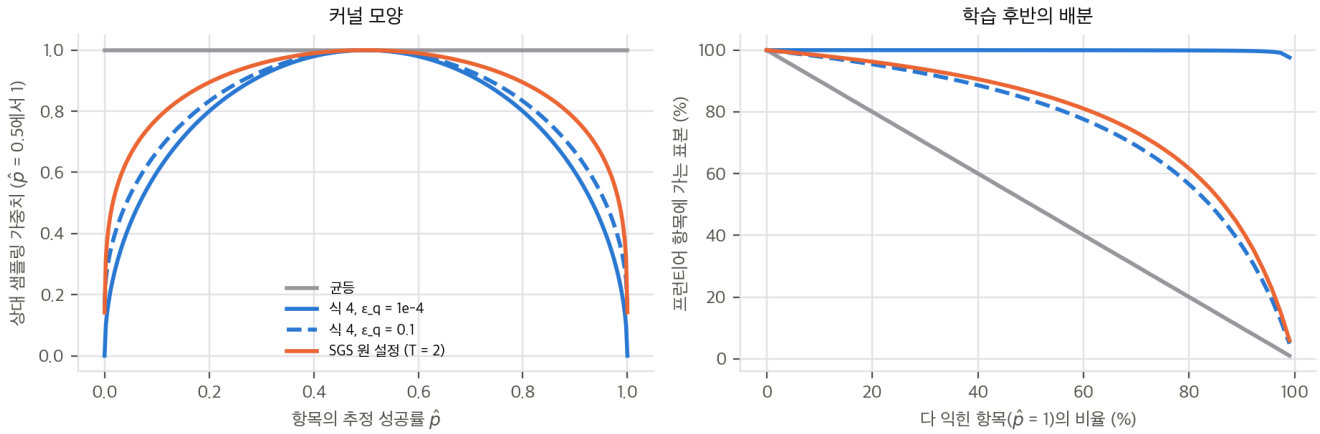
$$
\begin{aligned}
w &= (p+\varepsilon)^{\kappa t}\,(1-p+\varepsilon)^{\kappa(1-t)} \approx \sqrt{p(1-p)} \\
\ell &= \log(\max(w, \varepsilon) + \varepsilon) \\
q &\propto \exp(\ell / T) \approx (p(1-p))^{1/4}
\end{aligned}
$$

즉 SGS는 온도 2의 softmax 때문에 지수가 ¼이 됩니다. 식 4는 SGS에서 $T = 1$로 둔 것과 같습니다. AlphaNIST는 "SGS의 샘플링 규칙을 유지한다"고 쓰지만 두 커널은 같지 않습니다. 실제 구현이 SGS 코드를 그대로 썼다면 식 4가 부정확한 것이고, 식 4대로 구현했다면 SGS보다 탐욕적인 샘플러를 쓴 것입니다. 어느 쪽인지는 코드가 없어서 알 수 없습니다.

### 1.2 학습 단계별 배분 (S1, 실제 p를 안다고 가정)

$M = 32{,}768$개 항목입니다(SGS 조작 설정의 N과 같게 둠. AlphaNIST의 M은 미기재). 아래는 프런티어($0 < p < 1$) 항목에 가는 표본 비율입니다.

| bank 상태 | 균등 | 식 4, $\varepsilon_q$ = 1e-4 | 식 4, $\varepsilon_q$ = 1e-2 | 식 4, $\varepsilon_q$ = 0.1 | SGS (T = 2) |
|---|---|---|---|---|---|
| 초기: 80% 못 풂 | 20% | 99.9% | 91.5% | 56.5% | 61.6% |
| 중반: 30% 익힘, 30% 못 풂 | 40% | 100% | 96.6% | 77.6% | 81.1% |
| 후반: 95% 익힘 | 5% | 99.6% | 69.5% | 21.6% | 25.3% |




AlphaNIST 식 4와 SGS(arXiv:2610.12465 식 1-2, κ = 1, t = 0.5, ε = 1e-4, T = 2). AlphaNIST의 ε_q는 미기재.

해석:

- **ε_q가 정해지지 않으면 샘플러가 정해지지 않습니다.** 학습 후반의 프런티어 비율이 ε_q에 따라 22%에서 99.6%까지 달라집니다. ε_q = 0.1인 식 4가 SGS(T = 2)와 가장 비슷합니다.
- **"Beta가 균등보다 낫다"(그림 7)는 어떤 Beta인지에 달려 있습니다.** 다 익힌 항목이 대부분인 후반에, 균등은 표본의 95%를 이미 성공하는 시작점에 씁니다. 이진 보상에서 이런 episode는 advantage가 거의 0입니다. 표본당 이항 분산 E_q[p(1−p)]는 균등 0.009, SGS(T = 2) 0.048, 식 4(ε_q = 1e-4) 0.194입니다.

### 1.3 추정 잡음과 '굳는 항목' (S2, S3)

$\hat{p}$를 최근 L = 50개 결과의 평균으로 쓰면(독립 구현의 기본값), 실제 p = 0.95인 항목도 $0.95^{50}$ = 7.7% 확률로 $\hat{p}$ = 1이 됩니다. 이때 그 항목은 ε_q만 받습니다. 실제 p가 0.9-1인 항목의 19.3%가 이 상태였습니다.

64K 환경, iteration당 32 step, 평균 episode 100-375 step을 가정하면, 학습 후반에 다 익힌 항목 하나가 다시 뽑히는 간격은 다음과 같습니다.

| 커널 | iteration 간격 (episode 100 step) | iteration 간격 (episode 375 step) | 25K iteration 동안 방문 |
|---|---|---|---|
| 식 4, ε_q = 1e-4 | 336 | 1,265 | 20-74회 |
| 식 4, ε_q = 1e-2 | 5 | 19 | 1,346-5,016회 |
| SGS (T = 2) | 2 | 8 | 3,278-12,310회 |
| 균등 | 1.6 | 6 | 4,167-15,625회 |

ε_q가 작으면 "다 익혔다"고 판정한 시작점을 수백 iteration 동안 다시 보지 않습니다. 다중 과제 학습에서 다른 과제를 배우다 그 시작점을 잊어도(망각) 늦게 알아챕니다. ε_q를 적지 않은 것은 작은 누락이 아닙니다.

## 2. 보상 구조: 성공 시 종료 + 양의 보조 보상

### 2.1 유도

완료 직전 상태에서 매 step 보조 보상 s를 받는다고 하겠습니다. "지금 끝내면" 성공 보상 R을 받고 종료합니다. "n step 기다린 뒤 끝내면" 다음을 받습니다.

```
s(1 − γⁿ)/(1 − γ) + γⁿ R
```

두 값의 차이는 $(1-\gamma^n)[s/(1-\gamma)-R]$입니다. 따라서 **$s/(1-\gamma) > R$이면 남은 step 수와 상관없이 끝까지 미루는 편이 낫습니다.** 조건은 n과 무관합니다.

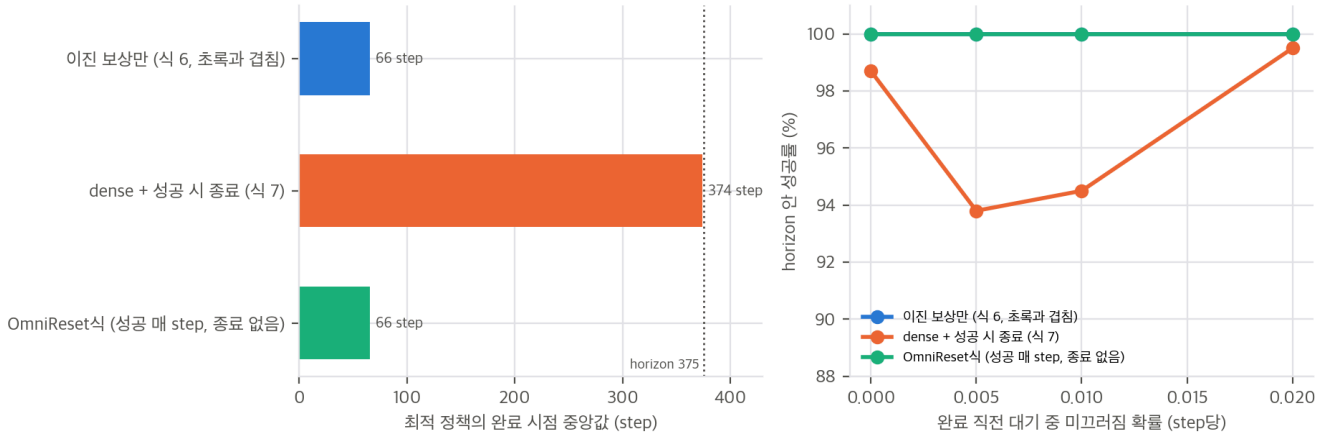
| γ | s = 0.01 | s = 0.02 |
|---|---|---|
| 0.99 | s/(1−γ) = 1.0 → 경계 | 2.0 > R = 1 → **미룸** |
| 0.995 | 2.0 → **미룸** | 4.0 → **미룸** |

AlphaNIST dense 비교군은 $c_2 = c_3 = 0.01$이라, 손이 부품 가까이 있고 방향이 맞으면 s가 최대 0.02입니다(두 항이 [0, 1] 범위라고 가정). 성공 보상 R = 1(식 5)이고 OmniReset의 γ = 0.99를 쓰면 미루는 쪽에 해당합니다.

OmniReset 구현은 이 문제를 구조로 피합니다. 성공 상태에 머무는 동안 매 step 성공 보상 1.0 × dt를 주고(실효 0.1/step, 보조 항 0.02/step보다 큼), 성공으로 종료하지 않습니다(`04` C14).

### 2.2 사슬 MDP로 확인 (H1, H2)

진행도 60칸, 전진 확률 0.9, horizon 375 step, γ = 0.99인 사슬 MDP를 backward induction으로 풀었습니다.




진행도 60칸 사슬 MDP, γ = 0.99, 전진 확률 0.9, backward induction(analysis/reward_toy.py). 미끄러짐이 크면(0.02) 최적 정책이 대기를 포기한다. 보조 항 꼴·γ·성공 보상 크기는 가정.

| 보상 | 완료 직전에서 끝내기를 처음 고르는 시각 | horizon 안 성공(미끄러짐 0) | 미끄러짐 1%/step |
|---|---|---|---|
| 이진 보상만 (식 6) | step 0 (즉시) | 100%, 중앙값 66 step | 100% |
| dense + 성공 시 종료 (식 7) | step 373 | 98.9%, 중앙값 374 step | 94.4% |
| dense + 성공 시 종료, R도 dt 배율(0.04)이면 | step 374 | 90.1% | 76.8% |
| OmniReset식 (성공 매 step, 종료 없음) | step 0 (즉시) | 100%, 중앙값 66 step | 100% |

해석:

- dense + 성공 시 종료에서 최적 정책은 **완료 직전에 머물다가 horizon 끝에 끝냅니다.** 확률적 전진 때문에 시간이 모자라 실패하는 경우가 생기고(1.1%), 머무는 동안 미끄러질 위험이 있으면 성공률이 더 떨어집니다.
- 학습 관점에서도 나쁩니다. 성공 신호가 episode 끝으로 밀리고, 정책은 정렬 근처에서 머무는 행동을 강화하게 됩니다.
- 이 toy로 공유 dense의 17.8%를 설명할 수는 없습니다. 분리 dense가 78%까지 간 것도 이 유인만으로는 설명되지 않습니다. 보조 항의 실제 꼴, γ, 성공 보상의 실제 스케일(Isaac Lab 보상은 dt를 곱함)이 논문에 없기 때문입니다. 다만 "이진 대 dense" 비교가 **보상 형식 외에 종료 구조의 차이까지 함께 바꾼 비교**라는 점은 분명합니다.

## 3. 그림 판독: reset 혼합 분포의 비중

그림 5(a) heatmap 80칸을 색 막대 역산으로 수치화했습니다. 행 평균(99.9 / 81.9 / 17.1 / 79.0)은 그림 5(b) 값(98.2 / 81.4 / 17.8 / 78.3)과 ±1.7%p(포화된 첫 행), 나머지 행은 ±0.7%p 안에서 맞습니다. 본문이 적은 두 칸(분리 dense M4 45.0%, 중간 기어 93.6%)도 45.6, 94.2로 재현됩니다.

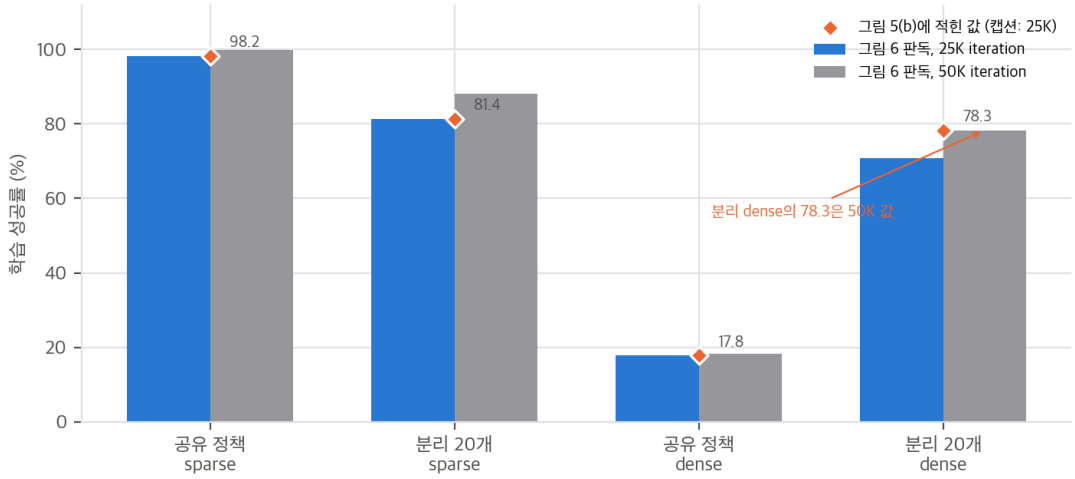
| 분리 sparse 정책 | heatmap 판독 | 본문 (무작위 초기 상태) |
|---|---|---|
| M4 너트 (N4) | 50.0% | ≤ 0.3% |
| 4 mm 막대 (R4) | 51.7% | ≤ 0.3% |
| 4 mm 페그 (P4) | 31.3% | ≤ 0.3% |

두 값이 모두 맞다면, heatmap은 모든 reset 종류의 평균이고 본문은 "무작위 초기 상태" 종류만의 값입니다. 분리 정책은 테이블에 놓인 부품을 집지 못하지만 손에 쥔 상태나 끼우기 직전 상태에서는 성공합니다. 그렇다면 이 과제들의 bank에서 그런 쉬운 시작점이 30-50% 정도를 차지한다고 볼 수 있습니다. 헤드라인 98.2%도 같은 혼합 분포의 평균입니다. 논문은 공유 정책이 테이블에 놓인 부품에서 시작했을 때의 성공률을 따로 보고하지 않습니다.

SGS 논문은 끼우기 직전 reset을 빼면 학습이 되지 않았다고 보고합니다(SGS 부록 B.3). 그리고 평가는 Reaching 초기 분포에서 따로 합니다(SGS 표 6). AlphaNIST에서도 쉬운 reset은 학습에 꼭 필요한 장치이지만, 평가 지표에 섞이면 과제의 실제 난이도를 가립니다.

## 4. 계산량 해부

- **처리량:** 64K 환경 × 25K iteration × 32 step = $5.12\times10^{10}$ step을 410 GPU-h에 썼습니다. GPU당 초당 약 34,700 step(환경당 8.7 step/s)이고, 16 GPU면 wall-clock 약 25.6시간입니다. GPU 종류는 적혀 있지 않습니다. FlashDexRetarget의 300M step과 비교하면 170배 많은 step입니다.
- **그림 5·6의 계산량 비율:**




판독: analysis/fig_digitize.py. 계산량은 같은 25K 기준이면 약 20배, 분리 정책을 50K까지 돌린 비용이면 39.3배(본문).

분리 정책 하나당 805 GPU-h는 공유 정책의 1.96배이고, 그림 6 곡선이 50K까지 이어진다는 점과 맞습니다. 같은 25K 기준이면 계산량 비율은 약 20배입니다(98.2% 대 81.4%). 분리 정책을 50K까지 돌린 비용이면 39.3배입니다(98.2% 대 88.1%). 본문의 "39.3배, 81.4%"는 앞의 성공률과 뒤의 비용을 짝지은 것입니다.

- **그림 7의 x축은 계산량 축입니다.** iteration을 고정했으므로 1K → 256K는 step 256배, GPU 1 → 64장입니다. 선형 가정에서 256K 실행은 약 1,640 GPU-h입니다.

## 5. Table I: 다중 부품 성공률의 구조

연속 비율 s_k / s_{k−1}은 0.977-0.989로 거의 일정합니다. s_k ≈ 0.99 × 0.9825^(k−1)로 적합되고, 10개에서 84%가 정확히 나옵니다.

- **독립 모델과의 차이:** 쌍마다 독립적으로 99%로 성공한다고 보면 10개에서 90.4%여야 합니다. 실제(84%)가 더 낮아서, 쌍이 늘수록 쌍당 실패가 1%에서 약 1.75%로 늘어납니다. 긴 horizon, 간섭, 순서 정하기의 비용으로 읽힙니다.
- **외삽:** 같은 비율로 20개를 외삽하면 70.8%이고, 10개 정책을 두 번 쓰면 70.6%입니다. 저자가 제안한 "10개씩 나눠 보드 전체 풀기"는 이 추세라면 70% 근처입니다.
- **빠진 정보:** 시행 수와 활성 쌍을 고르는 방법이 없어서 이 적합의 불확실성은 계산할 수 없습니다.

## 6. 망 차원 점검 (Fig. 3)

- 고유감각 한 프레임은 자세 7 + twist 6 + 관절 9 + 행동 8 = 30이고, 150 = 5 × 30이라 5프레임 이력으로 보입니다(본문에 이력 길이는 없음).
- 점군 임베딩 128 + 150 = 278로, 그림의 입력 차원과 맞습니다.
- SimBa 몸통(입력 사영 + residual block 2 + 출력)의 파라미터는 actor 약 113만, critic 약 112만입니다. 점군 인코더는 n_pts가 없어 셀 수 없습니다. 3 × n_pts를 그대로 펴서 256으로 사영한다면 n_pts = 1,000일 때 약 77만이 더해집니다(가설).

## 추가 검증 (2026-11-11)

### 추가 검증 1: 시작 상태 샘플러 × 보상 구조를 다중 과제 사슬 toy에서 비교

**목적.** §1(샘플러)과 §2(보상 구조)의 설명이 학습 속도와 최종 성공률로 드러나는지 본다. 식 4, SGS(T = 2), 균등 샘플러를 이진·dense+종료·OmniReset식 보상과 교차시키고, 결과가 §2.1의 "미루기" 유도와 맞는지 확인한다. 스크립트는 `analysis/extra_bank_grid.py`, 결과는 `analysis/extra_bank_grid_results.json`이다.

**설정.**

- 과제: 진행도 K = 20 칸의 사슬. 정책은 칸마다 시도 확률 σ(θ_k)를 가지며(표 형태), 시도가 성공하면 확률 0.9로 한 칸 전진한다. 과제 M = 200개가 시작 칸 0-19에 10개씩 배정된다. 시험 분포는 시작 칸의 균등이다.
- 학습: 매 iteration마다 샘플러가 과제 16개를 복원 추출하고, 벡터화된 에피소드를 T = 60 step 굴린다. REINFORCE(배치 평균 기준선), γ = 0.99, lr = 1.0, 1,500 iteration. 초기 θ = −2(시도 확률 0.12)이다. 이때 초기 정책의 시험 완료율은 0.32이고 칸 0은 0이라서 학습이 필요하다.
- 샘플러: 균등, 식 4($q \propto \varepsilon_q + \sqrt{\hat{p}(1-\hat{p})}$), ε_q = 1e-4, 1e-2, 1e-1), SGS(T = 2, κ = 1, t = 0.5, ε = 1e-4, $q \propto \exp(\ell/2)$). $\hat{p}$는 과제별 최근 50개 결과의 평균이고, 결과가 없으면 0.5다.
- 보상: 이진(완료 시 1, 종료), dense+종료(매 step 0.02·k/K, 완료 시 1, 종료), OmniReset식(매 step 0.02·k/K, 완료 상태에서 매 step 0.1, 종료 없음).
- 측정: 시작 칸별 horizon 안 완료 확률을 후진 DP로 정확히 계산해 평균한다(결정적). 25 iteration마다 기록하고, 학습 곡선의 평균(AUC)과 0.9 도달 시점을 본다. 완료 직전 칸(K−1)의 시도 확률 σ(θ_{K−1})도 본다. 시드 5개.

**결과.** 최종 성공률은 시험 분포 기준이고, AUC는 학습 곡선의 평균이다. p는 시드 5 대 5의 Mann-Whitney 검정(균등 대비)이다.

| 보상 | 샘플러 | 최종 성공률 | AUC | p (AUC, 대 균등) | 완료 직전 시도 확률 σ(θ_{K−1}) |
|---|---|---|---|---|---|
| 이진 | 균등 | 1.000 | 0.961 | - | 0.93 |

| 보상 | 샘플러 | 최종 성공률 | AUC | p (AUC, 대 균등) | 완료 직전 시도 확률 σ(θ_{K−1}) |
|---|---|---|---|---|---|
| 이진 | 식 4, ε = 1e-4 | 1.000 | 0.971 | 0.008 | 0.93 |
| 이진 | 식 4, ε = 1e-2 | 1.000 | 0.966 | 0.15 | 0.93 |
| 이진 | 식 4, ε = 1e-1 | 1.000 | 0.963 | 0.69 | 0.94 |
| 이진 | SGS (T = 2) | 1.000 | 0.964 | 0.55 | 0.93 |
| dense+종료 | 균등 | 0.956 ± 0.009 | 0.920 | - | 0.14 |
| dense+종료 | 식 4, ε = 1e-4 | 0.976 ± 0.008 | 0.947 | 0.008 | 0.16 |
| dense+종료 | 식 4, ε = 1e-2 | 0.982 ± 0.005 | 0.941 | 0.008 | 0.18 |
| dense+종료 | 식 4, ε = 1e-1 | 0.970 ± 0.011 | 0.936 | 0.008 | 0.15 |
| dense+종료 | SGS (T = 2) | 0.973 ± 0.006 | 0.938 | 0.008 | 0.15 |
| OmniReset식 | 균등 | 1.000 | 0.975 | - | 0.99 |
| OmniReset식 | 식 4, ε = 1e-4 | 1.000 | 0.982 | 0.032 | 0.99 |
| OmniReset식 | 식 4, ε = 1e-2 | 1.000 | 0.981 | 0.095 | 0.99 |
| OmniReset식 | 식 4, ε = 1e-1 | 1.000 | 0.979 | 0.095 | 0.99 |
| OmniReset식 | SGS (T = 2) | 1.000 | 0.979 | 0.095 | 0.99 |

0.9 도달 중앙값은 이진에서 125-150 iteration, OmniReset식에서 50-100 iteration이었다. dense+종료는 최종 성공률이 1에 못 미쳤다(0.96-0.98).

**결론.**

- 샘플러는 균등보다 학습을 빠르게 했다. 식 4와 SGS는 AUC 차이가 0.01 안쪽이었다. ε가 가장 작은 식 4(1e-4)가 세 보상 모두에서 가장 빨랐다. 이 toy에서 식 4와 SGS의 지수 차이(§1.1)가 학습 속도를 바꾸지는 않았다. 샘플러의 효과는 "균등이 아닌 프런티어 선택"에서 나온다.
- dense+종료에서는 정책이 완료 직전 칸에서 시도를 드물게 했다(σ(θ_{K−1}) ≈ 0.14-0.18, 이진은 0.93). 최종 성공률도 낮아졌다. 이것이 §2.1의 유도와 맞는다. s/(1−γ) = 0.02/0.01 = 2 > R = 1이면 완료를 미루는 쪽이 낫다. OmniReset식은 종료하지 않으므로 이 유인이 없었고, 학습도 가장 빨랐다.
- §1.3의 "굶는 항목"과 "잊음"은 이 toy에서 검증되지 않았다. 과제가 고정되어 있고 망각이 없으며, 과제당 표본이 약 120회씩 뽑혀 $\hat{p}$가 안정적이다. 따라서 ε가 작을 때 생기는 늦은 재방문의 해로움은 나타나지 않았다. 다중 과제 간섭과 망각을 넣은 실험이 필요하다.

**한계.**

- 표 형태 정책, 1차원 사슬, 과제 고정, REINFORCE(PPO가 아님). 논문의 Isaac Lab 학습과 다르다.
- 초기 θ = −2는 학습이 필요하도록 골랐다. 값에 따라 샘플러 간 차이가 달라질 수 있다.
- 시드 5개. 식 4 대 SGS처럼 차이가 작은 비교는 구분하기 어렵다.
- 보상의 크기와 종료 구조는 §2.1과 같은 기호로 두었지만, 실제 OmniReset 구현의 dt 배율은 반영하지 않았다.

# 04. 주장 검증: 수치 재계산, 구성 요소 코드 대조, 구현 가능성

- **코드 상태 (2026-10-28):** AlphaNIST 공식 코드는 공개되지 않았습니다("심사·수정 기간 뒤 공개 예정", Sec. I). `alpha-nist` 조직에는 웹사이트 저장소만 있습니다. SGS 페이지도 "Code (coming soon)"입니다. 그래서 논문 ↔ 코드 대조 대신 아래 세 가지를 했습니다.
  1. 논문 수치와 그림 판독값을 재계산했습니다.
  2. 논문이 가져왔다고 밝힌 구성 요소의 공개 원본과 대조했습니다(SGS 원문, OmniReset 구현 UW-Lab/UWLab @ `2b40afb`, Isaac Lab `reward_manager.py` @ `210ee79`).
  3. 독립 구현 하나를 읽었습니다(suniique/assembly-nist @ `eb5d15e`).
- **범위:** GPU 실행 없이 CPU 스크립트만 돌렸습니다(`03` 표 참조).

## 1. 주장별 재계산과 대조

| # | 주장 (출처) | 재계산·대조 | 판정 |
|---|---|---|---|
| C1 | 분리 정책 20개가 39.3배 더 많은 GPU-h (Sec. V-A) | 16,100 / 410 = 39.27 | ✅ 산술은 맞음 |
| C2 | "실행당 상호작용 예산을 맞춤" (Sec. V) | 분리 정책 하나당 805 GPU-h = 공유 실행의 1.96배. 그림 6 곡선이 50K까지 이어지는 것과 맞아, 분리 정책은 50K까지 돈 비용으로 보임. 같은 25K 기준이면 약 20배 | ⚠️ 비교 기준이 섞임 (가설: 비용 ∝ iteration) |
| C3 | 그림 5: "64K env, 25k iteration" | 그림 6 판독으로 공유 sparse 98.3, 분리 sparse 81.4, 공유 dense 17.9는 25K 값과 맞음. **분리 dense 78.3은 50K 값**(25K에서는 70.9) | ⚠️ 캡션과 불일치 |
| C4 | 분리 정책은 M4 너트, 4 mm 페그, 막대 삽입에서 ≤ 0.3% (Sec. V-A) | heatmap 판독은 50.0, 31.3, 51.7%. 본문은 "무작위 초기 상태에서"라는 조건이 붙어 있어 둘 다 맞다면 heatmap은 쉬운 reset이 섞인 평균 | ⚠️ 해석 주의 (`03` §3) |
| C5 | 공유 정책이 분리 정책보다 높음, "세 정밀 조립은 전문가가 거의 못 풂" (Fig. 5) | 판독: 분리 sparse 60% 미만 칸은 N4, R4, P4, RJ. 나머지 16개 평균은 90.4%이고, 같은 칸에서 공유 정책은 99.4-100 | ✅ 우위 확인 (RJ45도 57%라 "세 과제"보다 넓음) |
| C6 | 그림 5(b)의 ± (98.2 ± 0.986, 81.4 ± 1.732, 17.8 ± 0.010, 78.3 ± 0.573) | 이항 잡음으로 역산한 시드당 시행 수가 182, 505, **14,600,000**, 5,180으로 들쭉날쭉. 그림 7 기준(시드당 640회)이면 17.8%의 SD는 1.5% | ⚠️ 0.010은 단위 오기로 보임(가설). ±는 평가 시드 SD이고 학습 반복이 아님 |
| C7 | 64K env, 25K iteration, 410 GPU-h (Sec. V-A) | $5.12\times10^{10}$ step, GPU 16장, wall-clock 25.6 h, GPU당 34,700 step/s | 참고 (GPU 종류 미기재) |
| C8 | "sparse 보상은 병렬 경험에서 크게 이득" (Fig. 7) | iteration 고정이라 1K → 256K는 step 256배, GPU 1 → 64장, 선형 가정 시 26 → 1,640 GPU-h | ✅ 결론은 맞지만 계산량을 맞춘 비교 아님 |
| C9 | 256K에서 Beta 97.4% vs 균등 84.3% (Sec. V-B) | 차이 13.1%p, 점마다 3,200회라 이항 SE 0.3-0.6%p | ✅ 평가 잡음보다 큼 (학습 시드 1개로 보임) |

| # | 주장 (출처) | 재계산·대조 | 판정 |
|---|---|---|---|
| C10 | Table I: 1-10쌍 99 → 84% | 연속 비율 0.977-0.989, 기하 적합 0.9825/쌍. 독립 99% 모델이면 10쌍 90.4%. 20쌍 외삽 70.8%, 10쌍 정책 두 번 70.6% | 참고 (시행 수·SD·활성 쌍 선택 방법 없음) |
| C11 | Fig. 3 차원 | 고유감각 30 × 5 = 150, 128 + 150 = 278 일치. SimBa 몸통 actor 1,126,152 / critic 1,124,353 파라미터 | ✅ (이력 5프레임은 추정) |
| C12 | 독립 구현 README의 표 | 부품 종류별 값을 과제 수(4·4·3·4·5)로 가중하면 98.43 / 98.69 vs 보고 98.4 / 98.7 | ✅ 내부 일관 (성능 자체는 미실행) |
| C13 | SGS는 "작업별 보조 파지·그리퍼 shaping을 유지" (Sec. II-C) | SGS v1 부록 A.3.2의 조작 보상: r_succ(λ = 100), r_fail(λ = 1), 행동 크기·변화율·관절 속도 정규화뿐. 파지·그리퍼 shaping 항 없음 | ❌ 원문과 다름 (AlphaNIST가 다른 판본을 봤을 가능성) |
| C14 | dense 계수 $c_2 = c_3 = 0.01$은 "OmniReset의 계수와 같다" (Sec. V) | OmniReset 구현: `ee_asset_distance_tanh`, `dense_success_reward` 가중치 0.1, Isaac Lab이 dt를 곱함 (`reward_manager.py`: `.mul_(dt)`), OmniReset dt = 12 × 1/120 = 0.1 s → **실효 0.01** ✅. 단 OmniReset은 `success_reward`(가중치 1.0)를 정렬된 동안 매 step 주고, 종료 조건은 time_out과 abnormal_robot뿐(성공 종료 없음) | ⚠️ 계수는 같지만 보상 구조가 다름 (`03` §2) |
| C15 | "SGS의 샘플링 규칙을 유지" (Sec. II-A) | SGS 조작 설정(κ 1, t 0.5, T 2) → q ∝ (p(1–p))^¼. 식 4는 지수 ½(= SGS에서 T = 1) | ⚠️ 식이 다름. ε_q·L·M 미기재 (`03` §1) |
| C16 | PPO 설정 "32 step, 5 epoch, minibatch 4" (Sec. IV-E) | OmniReset `rsl_rl_cfg.py`와 정확히 같음(그쪽 γ 0.99, λ 0.95, lr 1e-4, clip 0.2, entropy 0.006) | 참고 (AlphaNIST의 나머지 값은 미기재) |

## 2. 재현에 필요한데 논문에 없는 정보

| 항목 | 논문 | 영향 |
|---|---|---|
| reset bank 크기 M, 종류별 비율, 생성 방법·비용 | 없음 ("OmniReset을 적용") | 성공률이 혼합 분포의 평균이라 결과 해석이 bank 구성에 달림(`03` §3) |
| ε_q, 이력 L | 없음 | 후반 배분이 22-99.6%로 달라짐(`03` §1) |
| γ, λ, 학습률, clip, entropy, 행동 분포 초기 표준편차 | 없음 | 독립 구현은 entropy와 초기 std를 바꾸고서야 정체를 벗어났다고 기록함 |
| 제어 비용 계수 c | 없음 | |
| n_pts, 점군 샘플링 방식 | 없음 | 점군 인코더 크기와 관측 해상도 |
| r_reach, r_align의 함수 꼴 | 없음 | dense 비교군의 유인 구조(`03` §2) |
| 관절별 행동 스케일 | 없음 (Δq 상대 목표만) | 독립 구현은 관절별 스케일 조정이 필수였다고 기록함 |
| 접촉 모델 설정(나사산 여유, sim dt) | 없음 | 독립 구현(MuJoCo)은 너트 0.2 mm 여유, 0.5 ms step이 필요했다고 기록함 |
| GPU 종류, 학습 시드 수 | 없음 | GPU-h 비교와 재현성 |
| 다중 부품 실험의 활성 쌍 선택 방법, 시행 수 | 없음 | Table I의 불확실성 |

## 3. 자료 정합성 요약

- **맞는 것:** 39.3배 산술, 그림 5(a)와 5(b)의 정합(판독 ±0.7%p), 그림 6의 25K 값과 그림 5(b)의 세 막대, 본문의 dense 칸 값(45.0%, 93.6%), 그림 7의 본문 수치, Fig. 3 차원, OmniReset 계수의 실효값(0.01).
- **어긋나는 것:**
  - 그림 5(b) 분리 dense 막대의 checkpoint(C3).
  - 39.3배의 비교 기준(C2).
  - "17.8 ± 0.010%"(C6).
  - SGS의 보상 서술(C13)과 샘플링 식(C15).
  - dense 비교군과 OmniReset 보상의 구조 차이(C14).
- **해석이 필요한 것:** heatmap과 본문 "≤ 0.3%"의 관계(C4). 헤드라인 98.2%가 reset 혼합 분포의 평균이라는 점.

## 4. 구현할 수 있는가

### 4.1 공식 코드를 그대로 쓰는 경우

**지금은 불가능합니다.** 코드가 공개되지 않았습니다. 공개되더라도 논문 규모는 64K 환경 = GPU 16장 × 약 26시간(410 GPU-h)이고, 256K = GPU 64장입니다. Isaac Lab(Linux, NVIDIA GPU)이 필요해 이 맥에서는 돌릴 수 없습니다.

### 4.2 독립 MuJoCo 구현(suniique/assembly-nist)을 쓰는 경우

- **장점:** MIT 라이선스이고 사전학습 checkpoint 5개(2.5K-12.5K iteration, 54 MB)가 있습니다. 시작 상태별 성공률을 보고하고, CPU 시뮬레이션(MuJoCo)이라 CUDA가 필요 없습니다.
- **평가:** 16 CPU 코어로 3,200 episode에 약 1시간입니다(README).
- **학습:** 기준 실행은 GPU learner 1장 + CPU worker 960개(약 30 노드) × 약 26시간입니다. README는 한 대에서 512 환경으로 작게 돌리는 방법도 적어 두었습니다.
- **이 맥의 장애물:** 접촉 플러그인(나사산·커넥터 SDF)이 x86-64 전용 빌드(`-mf16c`)라 Apple Silicon에서는 그대로 빌드되지 않습니다. half-precision 변환 부분을 ARM용으로 고쳐야 합니다(미확인). mujoco와 torch 설치도 필요합니다.
- **판단:** 사전학습 정책 평가는 **조건부 가능**합니다(플러그인 이식 + 설치 승인 필요, 맥 CPU 코어 수만큼 느림). 학습 재현은 이 맥 한 대로는 현실적이지 않습니다.

### 4.3 이 맥에서 바로 할 수 있는 것

| 작업 | 가능 여부 |
|---|---|
| 수치 재계산, 그림 판독, 샘플러·보상 toy | ✅ 완료 (numpy, 설치 없음) |
| SGS 커널과 식 4를 작은 학습 toy(예: 다중 과제 bandit, 사슬 MDP 묶음)에 넣어 비교 | ✅ 설치 없이 가능 |
| 독립 구현의 사전학습 checkpoint 평가 | 조건부 (플러그인 ARM 이식, mujoco·torch 설치) |
| Isaac Lab 기반 학습 | ❌ (CUDA 필요) |

### 4.4 아이디어를 다른 환경에 다시 구현하는 경우

| 구성 요소 | 난이도 | 메모 |
|---|---|---|
| 이진 보상 + 제어 비용 + 성공 시 종료 | 낮음 | 성공 판정(1 mm)만 정확하면 됩니다 |

| 구성 요소 | 난이도 | 메모 |
|---|---|---|
| Beta 샘플러 | 낮음 | 수십 줄입니다. ε_q, L, 커널 지수를 명시하고 프런티어 비율을 기록해야 합니다 |
| reset bank(놓인 부품 ~ 부분 조립) | 중간 | 과제마다 유효한 중간 상태를 만드는 것이 일입니다(조립 역순, 파지 샘플링). OmniReset 코드(UWLab)가 참고가 됩니다 |
| 점군 관측 + SimBa actor·critic | 중간 | Fig. 3 구조는 명확하지만 n_pts와 이력 길이는 정해야 합니다 |
| 나사산·기어 접촉 시뮬레이션 | 높음 | 성능을 좌우합니다. 독립 구현도 접촉 수정 전에는 너트 성공률이 0%였습니다 |
| 수만 환경 규모 PPO | 높음 | 이진 보상의 이득은 규모에 크게 의존합니다(1K 38.6% → 8K 86.9%, 그림 7). 작은 규모로는 결과가 재현되지 않을 수 있습니다 |

**요약:** 알고리즘 자체(이진 보상 + reset bank + Beta 샘플링 + PPO)는 단순해서 재구현이 쉽습니다. 이미 독립 구현이 공개되어 비슷한 성공률을 보고했습니다. 어려운 부분은 정밀 접촉 시뮬레이션과 대규모 병렬 계산입니다. 이 맥에서는 샘플러·보상 설계를 toy로 검증하는 단계까지가 현실적이고, 정책 평가는 독립 구현의 플러그인을 ARM으로 옮겨야 가능합니다.

# 02. 선행연구와 계보

번호 [n]은 AlphaNIST의 참고문헌 번호입니다. SGS[7]는 원문을, OmniReset[10]은 공개 구현(UW-Lab/UWLab @ `2b40afb`)의 보상·PPO 설정을 직접 확인했습니다. 나머지는 논문의 참고문헌 목록을 옮긴 것입니다.

## 1. 계보도

```
[게임: 결과 보상만으로 전략 발견]  AlphaGo [8] → AlphaGo Zero [9]          ← 논문의 동기

[조립 시뮬레이션과 학습]
  Factory [2] (RSS 2022, Isaac Gym 접촉) → IndustReal [29] (sim-to-real) → AutoMate [20] (전문가·범용 정책)
  → Refinery [25] (초기 조건 샘플링으로 미세조정) · ARCH [26] (계층형 하이브리드)
     공통점: horizon 단축, 미리 잡은 상태, 시연·하위 기술 지정 같은 구조를 둠

[대규모 on-policy RL에서 시작 상태 다루기]
  OmniReset [10] arXiv:2603.15789 (ICLR 2026)  다양한 reset bank, 고정 dense 목적
  → SGS [7] arXiv:2610.12465 (CoRL 2026)      성공률 중간인 시작 상태에 집중, 100만 env 이상, 과제마다 정책 하나
         |
         ▼
  ★ AlphaNIST: OmniReset bank + SGS 샘플링 + 공통 이진 보상 + 정책 하나로 20개 과제

[다중 과제 RL]  PCGrad [19], PopArt [21] ↔ "단순 공유 망도 충분하다" Kurin et al. [22]  ← AlphaNIST는 후자 쪽
[망 구조]       SimBa [28] (residual + LayerNorm, ICLR 2025)
[시뮬레이터]    Isaac Lab [11], PhysX [17], Newton [18], mjlab [12]
```

**차별점 요약:** 같은 그룹(UW·NVIDIA)의 두 선행 작업을 합친 형태입니다. OmniReset은 시작 상태의 **범위**를, SGS는 그 안에서의 **배분**을 다루었고, 둘 다 과제마다 정책을 따로 두었습니다. AlphaNIST의 새 요소는 ① 20개 과제를 정책 하나로 묶은 것, ② 과제별 shaping을 없애고 공통 이진 보상만 쓴 것, ③ 다중 부품 episode입니다.

## 2. 구성 요소별 출처

| 역할 | 출처 | 이 논문에서의 쓰임 | 확인 |
|---|---|---|---|
| reset bank | OmniReset [10] | 조립 정체 $e_i$를 함께 저장해 20개 과제로 확장 (Eq. 3) | 논문 |
| 시작 상태 샘플링 | SGS [7] | Eq. 4, "SGS의 규칙을 유지" | SGS 원문: Beta 커널 + 온도 T = 2 softmax → $(p(1-p))^{1/4}$. 식 4는 지수 ½ (`03` §1) |
| 공통 이진 보상 | SGS의 "도메인별 공유 보상"(성공 + 작은 정규화) | 성공 1, 1 mm 임계, 성공 시 종료 | SGS: $\lambda_{succ}$ = 100, 실패 −1, 행동·변화율·속도 정규화. AlphaNIST가 SGS를 "작업별 파지 shaping을 유지"한다고 한 것과 다름(`04` C13) |
| dense 비교군 계수 | OmniReset [10] | $c_2 = c_3 = 0.01$ | OmniReset 구현: 가중치 0.1 × Isaac Lab dt(0.1 s) = 실효 0.01. 단 성공 보상을 매 step 주고 성공으로 종료하지 않음(`04` C14) |
| PPO 설정 | (명시 없음) | 32 step, 5 epoch, minibatch 4 | OmniReset `rsl_rl_cfg.py`와 같음($\gamma$ 0.99, $\lambda$ 0.95, lr 1e-4도 같을 가능성, 미확인) |

| 역할 | 출처 | 이 논문에서의 쓰임 | 확인 |
|---|---|---|---|
| actor·critic 망 | SimBa [28], ResNet [27] | 폭 256, 확장 1024, block 2개, 분리 망 | 논문 Fig. 3 |
| 로봇·제어 | Franka + libfranka 강성·감쇠, IndustReal [29]·AutoMate [20]의 하드웨어 맞춤 시뮬레이션 | 25 Hz Δq 목표 + 그리퍼 | 논문 |
| 시뮬레이터 | Isaac Lab [11], Factory [2]의 접촉 | GPU 병렬 환경 | 논문 |
| 과제 | NIST ATB1 [1] | 20개 조립 쌍(너트 4, 기어 3, 막대 4, 페그 4, 커넥터 5) | 논문 Fig. 2, 5 |

## 3. 이 저장소의 다른 논문과의 관계

| 논문 | 관계 |
|---|---|
| **FlashDexRetarget** (`flashdexretarget/`) | 둘 다 "정책 하나를 이질적인 과제 모음 전체에 학습하면 각자 학습할 때보다 낫다"를 보입니다. FlashDexRetarget은 손-물체 시연 50-1,000개, AlphaNIST는 조립 과제 20개입니다. **방향은 반대**입니다. FlashDexRetarget은 off-policy(FlashSAC)로 바꿔 300M step에 끝내고 PPO는 거의 0이었습니다(Fig. 4d). AlphaNIST는 on-policy PPO를 64K-256K 환경, 512억-2,050억 step으로 밀어붙입니다. 시작 상태 쪽도 대조됩니다. FlashDexRetarget은 레퍼런스를 **균등하게** 뽑아 개선 여지를 남겼고 (`flashdexretarget/analysis/04_code_verification.md` D13), AlphaNIST는 성공률 기반 샘플링이 256K에서 +13.1%p를 냈습니다. 보조 장치도 다릅니다. FlashDexRetarget은 물체 보조 힘 커리큘럼과 dense 기하 보상을 쓰고, AlphaNIST는 이진 보상만 씁니다. 공통 교훈은 "다양한 시작 상태(레퍼런스 위 무작위 프레임, reset bank)가 학습을 가능하게 한다"는 것입니다. |
| **DDPG** (`ddpg/`) | AlphaNIST는 DDPG 계열(off-policy actor-critic)이 아니라 PPO입니다. 이진 보상, 긴 horizon, 수만 개 병렬 환경이라는 조건에서 off-policy를 쓰지 않은 이유는 논문에 없습니다. `ddpg` 분석의 교훈(구현마다 기본값이 다르니 설정을 명시할 것)이 여기서도 그대로 적용됩니다. AlphaNIST는 γ, λ, 학습률을 적지 않았습니다. |
| **DexScrew** (`dexscrew/`) | 둘 다 시뮬레이션 RL로 나사·너트 조작 기술을 배웁니다. DexScrew는 너트를 회전 관절로 단순화해 다지 손으로 돌리고, AlphaNIST는 나사산 접촉을 그대로 시뮬레이션해 평행 그리퍼로 체결합니다(한쪽 손끝 두드림 같은 전략이 나옴). DexScrew는 실제 텔레옵과 촉각 BC로 넘어갔고, AlphaNIST는 아직 시뮬레이션 단계입니다. |

## 4. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

| 주제 | 관련성 | 근거 수준 |
|---|---|---|
| **Reverse curriculum** (Florensa et al. [24], 논문이 인용) | 목표 근처에서 시작해 점점 멀리 가는 방식입니다. AlphaNIST의 reset bank + Beta 샘플링은 이것의 자동판으로 볼 수 있습니다. 쉬운 시작점이 많이 섞여 있다는 점(`03` §3)도 같은 맥락입니다. | 논문 인용 + 내 해석 |
| **Prioritized Level Replay, Sampling for Learnability(SFL)** | SGS가 직접 비교한 방법입니다(SGS 2절, 표 1). SFL은 p(1−p)를 쓰는데, AlphaNIST 식 4의 √(p(1−p))는 SFL과 SGS(T = 2)의 중간 지수입니다. | SGS 원문 |
| **POET, ACCEL 등 비지도 환경 설계** | 성공률 기반 배분의 상위 계보입니다. | 내 사전 지식, 서지 확인 필요 |
| **희소 보상 + 종료 설계**(정지 상태 보상, 종료 시 보상 처리) | 성공 시 종료에 매 step 양의 보조 보상을 붙이면 완료를 미루는 유인이 생깁니다. potential-based shaping(Ng et al., 1999)은 이 문제를 피하는 고전적 방법입니다. | 내 사전 지식(Ng et al. 은 서지 확인 필요) + `03` §2 toy |

## 5. 관련 GitHub 저장소 (2026-10-28 `gh api` 기준)

공식 코드는 아직 없습니다. AlphaNIST는 "심사·수정 기간 뒤 공개 예정"(Sec. I)이고, SGS 페이지는 "Code (coming soon)"입니다.

| 저장소 | 무엇인가 | 공식 여부 | 라이선스 | ★ | 마지막 push | 확인한 커밋 |
|---|---|---|---|---|---|---|
| alpha-nist/alpha-nist.github.io | 프로젝트 웹사이트(영상, 그림) | 공식, 코드 없음 | 표기 없음 | 1 | 2026-10-15 | `c568417` |
| SGS-RL/sgs-rl.github.io | SGS 프로젝트 웹사이트 | 공식, 코드 없음 | 미확인 | 8 | 2026-10-09 | `cb2423d` |
| UW-Lab/UWLab | OmniReset[10] 구현이 들어 있는 UW 연구실 Isaac Lab 확장 | 구성 요소 공식 | BSD-3-Clause | 340 | 2026-09-28 | `2b40afb` (보상·PPO·종료 설정 파일을 읽음) |
| isaac-sim/IsaacLab | 시뮬레이션 틀[11], Factory·AutoMate 환경 포함 | 구성 요소 공식 | BSD-3 계열 | 8,504 | 2026-10-28 | `210ee79` (`reward_manager.py` 의 dt 곱셈 확인) |
| newton-physics/newton | GPU 물리 엔진[18] | 관련 공식 | Apache-2.0 | 5,855 | 2026-10-28 | `d632fb7` (메타데이터만) |
| suniique/assembly-nist | **독립 MuJoCo 구현**: Franka, 20개 과제, 이진 보상, Beta 샘플링(식 4 그대로, ε_q = 1e-4, L = 50), SimBa식 망, 사전학습 checkpoint 5개 | 비공식 (저자와 무관하다고 명시) | MIT | 64 | 2026-10-23 | `eb5d15e` (clone, README·sampler·train 설정 확인) |
| ZoreAnuj/AlphaNIST | 비공식 구현. 실험은 UR7e와 다른 조립 과제(모터 마운트 등)로 바꿈 | 비공식 | 자체 문구 (README 참조) | 0 | 2026-10-22 | `e0eb350` (README만) |
| dcbenwu/alpha-nist-repro | README뿐인 재현 계획 저장소 | 비공식 | 표기 없음 | 3 | 2026-10-14 | `9551812` (README만) |

**독립 MuJoCo 구현(suniique/assembly-nist)에서 볼 점:**

- 8,192 환경, 10K PPO iteration에서 평가 성공 98.4%라고 보고합니다(AlphaNIST 프로토콜: 시드 5 × 과제당 32회, 확률적 행동). 시작 상태별 값도 있습니다. 12.5K iteration 기준으로 놓인 부품 98.1%, 집기 98.8%, 운반 98.8%, 부분 조립 99.0%, 끼우기 직전 98.7%입니다. README 표의 부품 종류별 값을 과제 수로 가중 평균하면 보고된 전체 값(98.4%, 98.7%)과 0.03%p 안에서 맞습니다(`04` C12). 시뮬레이터, 자산, 접촉 모델이 달라 AlphaNIST와 직접 비교할 수는 없다고 README도 적었습니다.
- 처음 시도에서 막힌 지점을 기록해 두었습니다.
  - 탐색 유지: 학습되는 표준편차 초기값 1.0, entropy 0.005로 바꾸자 정체가 풀렸습니다.
  - 손목 속도: 관절별 행동 스케일을 조정해야 15초 안에 방향을 바꿀 수 있었습니다.
  - 물리 수정: 너트에 반경 방향 0.2 mm 여유를 주고, 기어·나사산은 0.5 ms step을 썼습니다.
  - 논문에 없는 설정이 성능을 좌우한다는 증거입니다.

- 학습 자원: GPU learner 1장과 CPU worker 960개(약 30 노드)로 10K iteration에 약 26시간 걸렸습니다. 평가는 16 CPU 코어로 3,200 episode에 약 1시간입니다. 접촉 플러그인은 x86-64용(`-mf16c`)이라 이 맥(arm64)에서는 그대로 빌드되지 않습니다.

**한계:** 4절의 POET·ACCEL, Ng et al. 서지는 내 사전 지식이라 게시 전에 원문을 확인해야 합니다. 비공식 저장소의 성능 주장은 저자 자기보고이며 이 분석에서 실행해 확인하지 않았습니다.